

Estruturas de Dados Avançadas (EDA) – INF1010  
Departamento de Informática – PUC-Rio - 2023.1  
Tarefa 4 – Compressão de arquivo utilizando o algoritmo de Huffman

Lívia Lutz dos Santos - 2211055

Luiza Marcondes Paes Leme - 2210275

### Introdução:

- O objetivo do programa consiste em implementar a compressão da informação contida em um arquivo texto através do algoritmo de Huffman, gerando códigos menores para caracteres mais frequentes e vice-versa. Para isso, é necessário percorrer o texto do arquivo guardando em um array a quantidade de ocorrências de cada caractere e depois criar uma lista encadeada ordenada por ocorrência em ordem crescente. Em seguida, montar uma árvore binária com os elementos da lista, de forma que cada dois elementos são filhos de um nó que recebe a soma de suas ocorrências. Por fim, percorremos essa árvore para ter os códigos de compressão e reescrever o texto do arquivo utilizando-nos.

### Estrutura do programa:

Usamos apenas 1 header (t4.h) para definir as funções usadas e as structs No e Cod.

```
/* Lívia Lutz dos Santos - 2211055
Luiza Marcondes Paes Leme - 2210275
*/

typedef struct no No;
typedef struct cod Cod;

void imprimeArray(int ocorrencias[], int tam);

No* insereLista(No* lista, char c, int quant);

No* formaLista(int ocorrencias[], int tam);

void arvImprime(No* a);

void imprimeLista(No* l);

No* montaArv(No* lista);

No* montaArvzinha(No** plista);

No* insereArvLista(No* lista, No* arv);

int codifica(No* arv, unsigned char cod[], int ind, Cod vCodigos[], int indCod);

void imprimeCodifica(Cod* vCodigos, int numC);

Cod* buscaCod(Cod* vCodigos, char c);
```

No módulo t4.c, importamos o header (t4.h) e as bibliotecas <stdio.h> e <stdlib.h> para lidar com alocações dinâmicas e printf.

Na struct No, criamos um campo do tipo char para armazenar o caractere, outro campo do tipo int para armazenar sua ocorrência e 3 ponteiros para No (No\*) para guardar referências para o próximo nó (no caso da struct estar sendo usada numa lista encadeada) e para os nós à esquerda e à direita (no caso da struct estar sendo usada numa árvore binária). Na struct Cod, criamos um campo char para armazenar o caractere de cada nó, um array de unsigned char para guardar os bytes de cada caractere após a compressão da mensagem e um inteiro representando o tamanho do array contendo o código.

```
/* Struct usada para manipular o array, a lista encadeada e a árvore binária */  
  
struct no {  
    char a;  
    int ocorrencia;  
    No* dir;  
    No* esq;  
    No* prox;  
};  
  
/* Struct usada para guardar o código para compressão */  
  
struct cod {  
    char c;  
    unsigned char cod[8];  
    int tam;  
};
```

Na função imprimeArray, passamos como parâmetro o array que guarda a ocorrência de cada caractere e seu tamanho e imprimimos, para cada elemento do array, o caractere armazenado e sua ocorrência.

```
/* Função que imprime os caracteres e suas respectivss ocorrências no array */  
  
void imprimeArray(int ocorrencias[], int tam){  
    printf("Array:\n");  
    for (int i = 0; i < tam; i++) {  
        printf("'%' - %d ocorrencias\n", i, ocorrencias[i]);  
    }  
    return;  
}
```

Na função insereLista, passamos como parâmetro um ponteiro para Nó (correspondente a cabeça de uma lista encadeada), um caractere e um inteiro que corresponde a ocorrência do caractere. Com as informações do char e do int, criamos, por alocação dinâmica de memória, um novo nó da lista que guarda o char e sua ocorrência, caso a lista esteja vazia, o nó criado será o primeiro, senão, teremos que encontrar uma

posição adequada para inseri-lo na lista. Caso a lista tenha apenas 1 elemento, verificamos a ocorrência do novo nó, caso seja menor, ele será o cabeça da lista, senão ele será o segundo elemento. No caso da lista ter mais de 1 elemento, buscamos até a posição que tenha uma ocorrência maior e inserimos o novo nó na posição anterior. Após a inserção, a função retorna a lista atualizada.

OBS: Vale ressaltar que a lista encadeada está ordenada por prioridade em ordem crescente, ou seja, os nós com menores ocorrências ocupam as menores posições e vice-versa.

```
/* Função que insere caracteres e suas respectivas ocorrências numa lista encadeada de forma ordenada */
/* OBS: A ordenação obedece a seguinte regra:
Caracteres com menores ocorrências tem maior prioridade, enquanto os de maior ocorrência ficam no final da lista */
No* insereLista(No* lista, char c, int quant) {
    No* novo = NULL, * p = lista, * ant = NULL;

    /* Criamos um novo nó da lista */
    novo = (No*)malloc(sizeof(No));

    if (novo == NULL) {
        printf("Erro na alocação");
        return NULL;
    }

    /* inserimos suas informações */
    novo->a = c;
    novo->ocorrencia = quant;
    novo->dir = NULL;
    novo->esq = NULL;

    /* Caso a lista esteja vazia, esse nó será o primeiro */
    if (lista == NULL) {
        lista = novo;
        novo->prox = NULL;
        return lista;
    }

    /* Caso a lista tenha 1 elemento apenas e o novo tiver menos ocorrências, ele será o novo primeiro elemento */
    if (p->ocorrencia >= quant) {
        lista = novo;
    }
}
```

```
/* Procuramos a posição certa para inserir o novo elemento, guardando os nos anteriores e próximos */
else {
    for (; p != NULL; p = p->prox) {
        if (p->prox != NULL) {
            if (p->prox->ocorrencia >= quant) {
                ant = p;
                p = p->prox;
                break;
            }
        }
        ant = p;
    }
    ant->prox = novo;

    novo->prox = p;
    return lista;
}
```

Na função `formaLista`, passamos como parâmetro o array de inteiros guardando ocorrências de caracteres e seu tamanho. A função primeiro cria um ponteiro do tipo `No` nulo para inicializar a lista encadeada e para cada elemento do array, chamamos a função `insereLista` definida acima para inserir na lista os caracteres com ocorrência presentes no array, retornando a lista atualizada.

```

/* Função que insere cada elemento de um array de inteiros numa lista encadeada ordenada por prioridade */
No* formalista(int ocorrencias[], int tam) {
    No* lista = NULL;
    for (int i = 0; i < tam; i++) {
        /* inserimos apenas os elementos que tem ocorrência no array */
        if (ocorrencias[i] > 0)
            lista = insereLista(lista, i, ocorrencias[i]);
    }
    return lista;
}

```

Na função `imprimeLista`, passamos como parâmetro um ponteiro do tipo `no` que corresponde ao cabeça de uma lista encadeada. Até o último elemento da lista (ou até quando o ponteiro que usamos para percorrer a lista seja nulo), imprimimos o caractere e sua ocorrência contidos em cada nó da lista encadeada.

```

/*Função que imprime os caracteres e a ocorrência de cada no de uma lista encadeada ordenada por prioridade */
void imprimeLista(No* lista) {
    printf("\nLista encadeada por prioridade:\n\n");
    for (No* p = lista; p != NULL; p = p->prox) {
        printf("%c - %d ocorrencias\n", p->a, p->ocorrencia);
    }
    return;
}

```

Na função `montaArvzinha`, passamos como parâmetro um ponteiro para um ponteiro do tipo `no`. Caso o ponteiro do tipo nó (conteúdo do parâmetro) seja nulo, a lista encadeada está vazia e a função retorna `NULL`. Caso contrário, guardamos os próximos 2 nós da lista em 2 variáveis do tipo nó. Criamos um novo nó por alocação dinâmica que guardará o caractere '+' (indicando a soma das ocorrências dos nós à esquerda e à direita, valor que será armazenado no campo inteiro do nó criado). As variáveis guardando os ponteiros para os próximos nós do ponteiro `No` do parâmetro serão, agora, os nós esquerdo e direito, respectivamente, do novo nó criado (esse ponteiro `No` criado será usado como árvore binária). Guardamos a referência para o cabeça da lista em outra variável (que será passada por referência para a função que a chamou) e definimos campo próximo da árvore como `NULL`. A função retorna a árvore binária formada a partir do nó da lista.

```

/* Função que monta uma árvore binária a partir de uma lista encadeada ordenada por prioridade */
No* montaArvzinha(No** plista) {
    No* p1 = *(plista), * p2, * p3, * arv;

    /* Caso de lista vazia */
    if (p1 == NULL) {
        return NULL;
    }

    /*Guardando os próximos 2 nós da lista */
    p2 = p1->prox;
    p3 = p2->prox;

    arv = (No*)malloc(sizeof(No));
    if (arv == NULL)
        return NULL;

    /* Cada nó será a soma das ocorrências de seus 2 filhos */
    arv->a = '+';

    arv->ocorrencia = p1->ocorrencia + p2->ocorrencia;

    arv->esq = p1;

    arv->dir = p2;

    arv->prox = NULL;

    *(plista) = p3;

    return arv;
}

```

Na função `montaArv`, passamos como parâmetro o cabeça de uma lista encadeada e para cada elemento da lista encadeada, chamamos a função `montaArvzinha` para criar uma árvore binária a partir desses nós. Quando chegamos no final da lista, atualizamos a lista encadeada inserindo a nova árvore binária formada na lista e a função retorna a lista atualizada.

```

/*Função que insere árvores binárias numa lista encadeada ordenada por prioridade */
No* montaArv(No* lista) {
    No** plista = &lista, *arv;

    /*Para cada nó da lista, formamos uma árvore a partir dele e o inserimos na lista*/
    while (lista->prox != NULL) {
        arv = montaArvzinha(plista);
        if (arv == NULL)
            printf("Erro ao montar arvore");
        lista = insereArvLista(lista, arv);
    }

    return lista;
}

```

Na função `insereArvLista`, passamos como parâmetro um ponteiro do tipo `No` que corresponde a cabeça de uma lista encadeada e outro ponteiro do tipo `No` que corresponde à raiz de uma árvore binária. Caso a lista esteja vazia, inserimos a árvore como primeiro elemento retornando a lista atualizada. Se a lista tiver apenas 1 elemento, verificamos a ordenação e inserimos a árvore retornando a lista atualizada. Caso a lista tenha mais de 1 elemento, achamos a posição ideal para inseri-la de acordo com a ordenação e retornamos a lista atualizada.

```

/* Função auxiliar que insere uma árvore binária como nó de uma lista encadeada */
No* insereArvLista(No* lista, No* arv) {
    No* p = lista, * ant = NULL;

    /* Caso a lista esteja vazia, inserimos a árvore como primeiro nó */
    if (lista == NULL) {
        lista = arv;
        return lista;
    }

    /* Caso se ser o primeiro elemento */
    if (p->prox == NULL) {

        /* Ordenando por prioridade */
        if (p->ocorrenciam >= arv->ocorrenciam) {
            lista = arv;
            arv->prox = p;
        }
        else
            p->prox = arv;
        return lista;
    }
}

```

```

/* Buscamos a posição ideal para inserir o nó da árvore na lista */

else {
    for (; p != NULL; p = p->prox) {
        if (p->prox != NULL) {
            if (p->prox->ocorrenciam >= arv->ocorrenciam) {
                ant = p;
                p = p->prox;
                break;
            }
        }
        ant = p;
    }
    ant->prox = arv;

    arv->prox = p;
    return lista;
}

```

Na função arvImprime, passamos como parâmetro um ponteiro do tipo nó que equivale à raiz da árvore e para cada nó dela, imprimimos seu char e sua ocorrência no percurso pré-ordem.

```

/*arvImprime imprime a árvore binária em pré- ordem, ou seja, primeiro as informações no nó raiz,
seguido dos nós à esquerda e por fim os nós à direita*/
void arvImprime(No* a) {
    /*Até o final da árvore, imprimimos suas informações*/

    if (a != NULL) {

        printf("Char: '%c' - ocorrencia: %d\n", a->a, a->ocorrenciam);

        /*Imprime as informações dos nós à esquerda recursivamente*/
        arvImprime(a->esq);

        /*Imprime as informações dos nós à direita recursivamente*/
        arvImprime(a->dir);
    }
}

```

Na função `codifica`, passamos como parâmetros um ponteiro do tipo `No` para árvore binária, um array de bytes (`unsigned char`) para guardar os bytes de cada caractere codificado, um inteiro `ind` para guardar o índice do array de inteiros, um array do tipo `Cod` para armazenar os caracteres e suas codificações e um inteiro `indCod` para guardar o índice do array do tipo `Cod`. Caso o nó de árvore seja uma folha, guardamos seu caractere no array `Cod` no índice presente, caso não seja uma folha, repetimos o processo nas sub-árvores esquerda e direita, porém quando percorremos a esquerda, sua codificação no array de inteiros será 0 e à direita será 1, retornando o tamanho do array de tipo `Cod`.

```
/* Função que percorre a árvore e gera os códigos */
int codifica(No* arv, unsigned char cod[], int ind, Cod vCodigos[], int indCod) {

    /*Se o nó for folha, e um caracter, logo o inserimos em vCodigos*/

    if (arv->esq == NULL && arv->dir == NULL) {
        Cod* codificado = (Cod*)malloc(sizeof(Cod));
        if (codificado == NULL) {
            printf("Erro na alocação");
            exit(1);
        }
        codificado->c = arv->a;
        codificado->tam = ind;
        for (int i = 0; i < ind; i++) {
            codificado->cod[i] = cod[i];
        }
        vCodigos[indCod] = *codificado;
        indCod++;
    }

    /*Repetimos o processo para a subárvore esquerda e direita*/

    if (arv->esq != NULL) {
        cod[ind] = 0;
        indCod = codifica(arv->esq, cod, ind + 1, vCodigos, indCod);
    }
    if (arv->dir != NULL) {
        cod[ind] = 1;
        indCod = codifica(arv->dir, cod, ind + 1, vCodigos, indCod);
    }
    return indCod;
}
```

Na função `imprimeCodifica` passamos como parâmetro um array do tipo código e um inteiro contendo seu tamanho. Para cada índice do array, imprimimos seu caractere e seu valor codificado.

```
/* Função que imprime cada caracter codificado em binário*/
void imprimeCodifica(Cod* vCodigos, int numC) {
    printf("\nCodigos:\n");

    for (int i = 0; i < numC; i++) {
        printf("\n'%c' - ", vCodigos[i].c);

        for (int k = 0; k < vCodigos[i].tam; k++)
            printf("%d", vCodigos[i].cod[k]);
    }
}
```

Na função `buscaCod`, passamos como parâmetro um array do tipo `Cod` e um caractere e assim, percorremos o array até encontrarmos o caractere no parâmetro, retornando um ponteiro `Cod` com suas informações.

```

Cod* buscaCod(Cod* vCodigos, char c){
    Cod* p = vCodigos;

    if (p == NULL) {
        return NULL;
    }

    while (p->c != c) {
        p++;
    }

    return p;
}

```

### Solução:

Na main, declaramos duas variáveis de ponteiro para arquivo para lermos o arquivo com a mensagem a ser codificada e para escrevermos a mesma em um arquivo binário, uma variável do tipo unsigned char para guardar cada caractere lido da mensagem contida no arquivo, 2 ponteiros do tipo nó para serem usados na criação da lista encadeada e da árvore binária, variáveis inteiras para fazer a contagem de número de caracteres, índices de arrays, número de letras, número de bytes escritos, posição em um array e quantidade de bits em cada byte de um array, 2 variáveis do tipo ponteiro para Cod para guardar informações da mensagem codificada. Também inicializamos o array de inteiros de tamanho 128 que conta a ocorrência de cada caractere na mensagem com zeros e um array de bytes de tamanho 8 com alocação estática para guardar os bytes a serem escritos no arquivo binário.

```

int main(void) {
    FILE* f,*b;
    unsigned char c;
    No* lista, * arvore;
    int numC = 0, indCod = 0, numL = 0, gravado = 0, pos = 0, bit = 0;
    Cod* vCodigos,*p;
    unsigned char cod[8];

    //Array que contém o número de ocorrências de cada char no arquivo
    //Inicializa o array com 0

    int ocorrencias[128] = { 0 };
}

```

Usamos o ponteiro FILE para abrir o arquivo da mensagem a ser codificada para leitura e imprimimos uma mensagem de erro caso seja nulo. Assim, começamos a leitura até o fim do arquivo lendo cada caractere da mensagem e incrementando 1 na posição correspondente de cada caractere no array de ocorrências, sendo o índice de cada caractere no array corresponde ao seu código ASCII e com isso fechamos o arquivo. Porém, antes disso, também contamos a quantidade de letras presentes no arquivo para usar futuramente quando formos escrever os bytes codificados no arquivo binário. Após



isso, imprimimos o caractere e a ocorrência de cada um deles no array chamando a função `imprimeArray`.

Com isso, podemos montar a lista encadeada de caracteres a partir do array chamando a função `formaLista` e guardando seu resultado numa variável de ponteiro para tipo `No`, então chamamos a função `imprimeLista` para exibir os nós da lista encadeada. Após isso, fazemos um loop para contar a quantidade de nós que formam a lista encadeada para saber quantos caracteres codificados serão.

```
//Abre o arquivo

f = fopen("frase.txt", "r");

if (f == NULL) {
    printf("erro ao abrir arquivo\n");
    return -1;
}

/*Lê os caracteres do arquivo e de acordo com seu valor ASCII(indice no array)
* e incrementa 1 toda vez q esse caracter for encontrado, também contamos o número de letras
(caracteres) presentes nesse arquivo */

while (!feof(f)) {
    fread(&c, sizeof(unsigned char), 1, f);
    ocorrencias[c]++;
    numL++;
}

/* Fechando o arquivo */

fclose(f);

//Imprimindo o array de caracteres resultante

imprimeArray(ocorrencias, 128);

/*Criando a lista encadeada ordenada por prioridade(menos->mais ocorrências)
* para os caracteres do array */

lista = formaLista(ocorrencias, 128);

//Imprimindo a lista resultante

imprimeLista(lista);

for (No* p = lista; p != NULL; p = p->prox) {
    numC++;
}
```

Agora que temos a lista encadeada formada, podemos montar a árvore binária a partir dos seus nós, para isso, chamamos a função `montaArv` e guardamos seu resultado num outro ponteiro para tipo `nó`. Em seguida, chamamos a função `arvImprime`, que imprime as informações contidas em cada nó da árvore binária no percurso pré-ordem.

OBS: Usamos a mesma struct para manipular a árvore binária e a lista encadeada, por isso os ponteiros são do mesmo tipo.

Já que temos a árvore formada, começamos, então, a codificar a mensagem. Para isso, inicializamos um array do tipo `Cod` fazendo alocação dinâmica para que esse array tenha o tamanho necessário para guardar a quantidade de nós existentes. Após isso, chamamos a função `codifica` para codificar os caracteres em cada nó e inserir no array do tipo `Cod` para então chamar a função `imprimeCodifica` para imprimir o array do tipo `Cod` formado com os caracteres e sua codificação.

```

/*Montando a árvore binária */
arvore = montaArv(lista);

/* Imprimindo a árvore binária usando o percurso pre-ordem */
printf("\nArvore em pre-ordem:\n\n");
arvImprime(arvore);

/*Array de codigos para guardar os caracteres codificados*/
vCodigos = (Cod*)malloc(numC * sizeof(Cod));
if (vCodigos == NULL)
    return -1;

indCod = codifica(arvore, cod, 0, vCodigos, indCod);

/*Imprimindo cada caractere e sua codificacao*/
imprimeCodifica(vCodigos, numC);

```

Após a mensagem codificada ser exibida, começaremos a ler a mensagem original do arquivo, gerando a mensagem codificada a partir da sua leitura. Enquanto a leitura acontece, iremos criar um vetor de bytes para guardar a codificação de cada letra para escrevermos no arquivo binário, iniciando seus campos com zero no começo.

```

/* Lendo cada caracter do arquivo e imprimindo a mensagem comprimida*/
f = fopen("frase.txt", "r");

if (f == NULL) {
    printf("erro ao abrir arquivo\n");
    return -1;
}

/*Cria um array de bytes a partir da mensagem comprimida para escrever no arquivo binário*/
unsigned char *vComprimido = (unsigned char*)malloc(sizeof(unsigned char) * numL);
if (vComprimido == NULL) {
    printf("Erro na alocao\n");
    exit(1);
}

/* Inicializando o array com zeros*/
for (int i = 0; i < numL; i++)
    vComprimido[i] = 0;

```

Conforme a leitura acontece, buscamos o código do caracter lido no array vCodigos e para cada um deles, adicionamos seus bits no array vComprimido e caso o índice desse array esteja já preenchido com 8 bits, passamos a preencher os próximos índices da mesma forma, sempre contando a quantidade de bits preenchidos em cada índice. No final da leitura, ainda acrescentamos uma condição para exibir bits que podem não ter sido exibidos durante a leitura, fechando o arquivo texto em seguida.

```

/*Lendo a mensagem comprimida do arquivo*/
printf("\n\nMensagem comprimida:\n");
while (!feof(f)) {
    fread(&c, sizeof(unsigned char), 1, f);

    p = buscaCod(vCodigos, c);

    /*Montando o array de bytes a partir da mensagem comprimida para ser escrito no arquivo binário*/
    for (int i = 0; i < p->tam; i++) {
        printf("%d", p->cod[i]);

        /*Adicionamos a parte comprimida em cada byte*/
        vComprimido[pos] |= p->cod[i];

        /* Caso seja o último bit da posição do array, exibimos esse byte e zeramos a quantidade de bits ocupados*/
        if (bit == 7) {
            printf("\nByte: 0x%02X\n\n", vComprimido[pos]);
            pos++;
            bit = 0;
        }

        /*Senão, abrimos espaço de 1 bit na posição e aumentamos a contagem de bits ocupados*/
        else {
            vComprimido[pos] <<= 1;
            bit++;
        }
    }
}

/*Caso para exibir o último byte do array*/
if (bit > 0) {
    printf("\nByte: 0x%02X\n\n", vComprimido[pos]);
    pos++;
}

/* Fechando o arquivo */
fclose(f);

```

Por fim, escrevemos a mensagem codificada por bytes no arquivo binário, para isso, usamos o ponteiro do tipo FILE\* para abrir um arquivo .dat, exibindo uma mensagem de erro no caso do arquivo não abrir. Depois, para cada posição do vetor de bytes, escrevemos seu conteúdo no arquivo binário, contando e exibindo quantos bytes foram gravados nele, fechando o arquivo logo em seguida.

```

/* Escrevendo o array de bytes no arquivo binario*/

b = fopen("comprimido.dat", "wb");

if (b == NULL) {
    printf("Erro ao abrir arquivo binario\n");
    exit(1);
}

gravado = fwrite(&vComprimido[0], sizeof(unsigned char), pos, b);

/*Imprimindo a quantidade de bytes gravados no arquivo*/
printf("\nBytes gravados no arquivo: %d", gravado);

/*Fechando o arquivo binário*/

fclose(b);

return 0;

```

**Saída do programa:**

[illegible]

|                      |                                 |                          |
|----------------------|---------------------------------|--------------------------|
| 'V' - 0 ocorrencias  | 't' - 4 ocorrencias             | 's' - 8 ocorrencias      |
| 'W' - 0 ocorrencias  | 'u' - 4 ocorrencias             | 'e' - 9 ocorrencias      |
| 'X' - 0 ocorrencias  | 'v' - 0 ocorrencias             | ' ' - 13 ocorrencias     |
| 'Y' - 0 ocorrencias  | 'w' - 0 ocorrencias             | 'a' - 15 ocorrencias     |
| 'Z' - 0 ocorrencias  | 'x' - 0 ocorrencias             |                          |
| '[' - 0 ocorrencias  | 'y' - 0 ocorrencias             | Arvore em pre-ordem:     |
| '\' - 0 ocorrencias  | 'z' - 1 ocorrencias             | Char:'+' - ocorrencia:98 |
| ']' - 0 ocorrencias  | '{' - 0 ocorrencias             | Char:'+' - ocorrencia:40 |
| '^' - 0 ocorrencias  | ' ' - 0 ocorrencias             | Char:'+' - ocorrencia:18 |
| '_' - 0 ocorrencias  | '}' - 0 ocorrencias             | Char:'+' - ocorrencia:9  |
| '\'' - 0 ocorrencias | '~' - 0 ocorrencias             | Char:'c' - ocorrencia:4  |
| 'a' - 15 ocorrencias | ' ' - 0 ocorrencias             | Char:'+' - ocorrencia:5  |
| 'b' - 0 ocorrencias  |                                 | Char:'p' - ocorrencia:2  |
| 'c' - 4 ocorrencias  | Lista encadeada por prioridade: | Char:'m' - ocorrencia:3  |
| 'd' - 5 ocorrencias  |                                 | Char:'e' - ocorrencia:9  |
| 'e' - 9 ocorrencias  | 'z' - 1 ocorrencias             | Char:'+' - ocorrencia:22 |
| 'f' - 3 ocorrencias  | 'l' - 1 ocorrencias             | Char:'+' - ocorrencia:10 |
| 'g' - 1 ocorrencias  | 'g' - 1 ocorrencias             | Char:'r' - ocorrencia:5  |
| 'h' - 0 ocorrencias  | 'A' - 1 ocorrencias             | Char:'d' - ocorrencia:5  |
| 'i' - 6 ocorrencias  | 'p' - 2 ocorrencias             | Char:'+' - ocorrencia:12 |
| 'j' - 0 ocorrencias  | 'm' - 3 ocorrencias             | Char:'n' - ocorrencia:6  |
| 'k' - 0 ocorrencias  | 'f' - 3 ocorrencias             | Char:'i' - ocorrencia:6  |
| 'l' - 1 ocorrencias  | 'u' - 4 ocorrencias             | Char:'+' - ocorrencia:58 |
| 'm' - 3 ocorrencias  | 't' - 4 ocorrencias             | Char:'+' - ocorrencia:27 |
| 'n' - 6 ocorrencias  | 'c' - 4 ocorrencias             | Char:' ' - ocorrencia:13 |
| 'o' - 7 ocorrencias  | 'r' - 5 ocorrencias             | Char:'+' - ocorrencia:14 |
| 'p' - 2 ocorrencias  | 'd' - 5 ocorrencias             | Char:'+' - ocorrencia:7  |
| 'q' - 0 ocorrencias  | 'n' - 6 ocorrencias             | Char:'f' - ocorrencia:3  |
| 'r' - 5 ocorrencias  | 'i' - 6 ocorrencias             | Char:'+' - ocorrencia:4  |
| 's' - 8 ocorrencias  | 'o' - 7 ocorrencias             |                          |

```
Char: '+' - ocorrencia:2
Char: 'z' - ocorrencia:1
Char: 'l' - ocorrencia:1
Char: '+' - ocorrencia:2
Char: 'g' - ocorrencia:1
Char: 'A' - ocorrencia:1
Char: 'o' - ocorrencia:7
Char: '+' - ocorrencia:31
Char: 'a' - ocorrencia:15
Char: '+' - ocorrencia:16
Char: '+' - ocorrencia:8
Char: 'u' - ocorrencia:4
Char: 't' - ocorrencia:4
Char: 's' - ocorrencia:8
```

```
Codigos:

'c' - 0000
'p' - 00010
'm' - 00011
'e' - 001
'r' - 0100
'd' - 0101
'n' - 0110
'i' - 0111
' ' - 100
'f' - 10100
'z' - 1010100
'l' - 1010101
'g' - 1010110
'A' - 1010111
'o' - 1011
'a' - 110
'u' - 11100
't' - 11101
's' - 1111
```

```
Mensagem comprimida:
10101111
Byte: 0xAF

11110000
Byte: 0xF0

11111111
Byte: 0xFF

01010011
Byte: 0x53

10011101
Byte: 0x9D

11100010
Byte: 0xE2

01101111
Byte: 0x6F

10001010
Byte: 0x8A

01100010
Byte: 0x62

11100101
Byte: 0xE5

10111111
Byte: 0xBF

10011111
Byte: 0x9F

10101110
Byte: 0xAE
```

```
01010011
Byte: 0x53

10001100
Byte: 0x8C

10111000
Byte: 0xB8

01100101
Byte: 0x65

10111011
Byte: 0xBB

10011111
Byte: 0x9F

11100000
Byte: 0xE0

10110010
Byte: 0xB2

01101001
Byte: 0x69

10100101
Byte: 0xA5

10100101
Byte: 0xA5

01101100
Byte: 0x6C

11001111
Byte: 0xCF
```

```
01010011
Byte: 0x53

00000110
Byte: 0x06

10111000
Byte: 0xB8

01100000
Byte: 0x60

11110011
Byte: 0xF3

00111000
Byte: 0x38

10111001
Byte: 0xB9

01010111
Byte: 0x57

00000110
Byte: 0x06

10111000
Byte: 0xB8

01101000
Byte: 0x68

11100000
Byte: 0xE0

11100101
Byte: 0xE5
```

```
10111010
Byte: 0xBA

01100010
Byte: 0x62

10011000
Byte: 0x98

11101101
Byte: 0xED

01001011
Byte: 0x4B

01000001
Byte: 0x41

11100000
Byte: 0xE0

10110011
Byte: 0xB3

11111111
Byte: 0xFE

Bytes gravados no arquivo: 48
```

## **Conclusão:**

O código não possui erros aparentes, porém tivemos um pequeno problema em relação às structs pois tivemos que colocar sua definição no arquivo principal já que apesar delas estarem definidas no .h e no módulo contendo o .h, quando o .h foi incluído no arquivo principal, esses tipos das structs não foram reconhecidos ("incomplete type is not allowed"). Também tivemos que alterar a mensagem no arquivo já que caracteres do tipo 'ã,ç,õ' não estão presentes na tabela ASCII e por isso a mensagem codificada não possui acentos.

Em relação ao desenvolvimento do programa, não tivemos maiores problemas, apenas a função de codificação que foi um pouco complexa de programar já que precisamos de estruturas externas para manipular a codificação e 2 variáveis diferentes para manipular os índices de cada array nesse processo de codificação.